

COMP3438 System Programming

Unix Processes

Course Organization

Part I: Unix System Programming (Device Driver Development)

Character Device Driver
Development

Introduction to Block
Device Driver

Overview of Device Driver Development

Process

File System

Overview of Unix Systems Programming

Part II: Compiler Design

Syntax Analysis

Lexical Analysis

Overview of Compiler Design

Overview of the Subject

Unix Processes

What is a process?

What does a process look like in the system?

When is a process created? By whom?

How is a process created? In how many ways?

When does a process stop? Can we wait for a process to die?

What is a process called if it never die?

Unix Processes: common definition

A process is an instance of a program in execution (the execution of the program has started but has not yet terminated).

Process is dynamic while program is static.

A process is the basic unit for competing the resources. In particular, it is the basic active entity to CPU scheduler.

Unix Processes: how to understand it?

State of a computer at clock tick (i.e. discrete time) i :

$S(i) = (\text{register 1's value, register 2's value, ...,}$
1st memory byte's value, 2nd memory byte's value, ...,
1st hard disk byte's value, 2nd hard disk byte's value, ...,
peripheral 1's register 1's value, peripheral 1's register 2's value, ...).

State of an execution e at clock tick i :

$s(e, i) = (\text{resources used by } e \text{ at } i, \text{ state of the resources used by } e \text{ at } i).$

A **process** is the time sequence of $\{s(e, i)\}$, where every two consecutive states $s(e, j)$ and $s(e, j+1)$ have causal relationship determined by the program logic and the OS.

Unix Processes: how to turn a program into a process?

The program is read into memory.

A unique process ID is assigned.

OS kernel creates a process structure instance to record information related to this process.

Necessary resources to run the program are allocated.

The initial state $s(e, 0)$ is set, which will trigger its next state $s(e, 1)$, which will trigger its next state $s(e, 2)$, ... determined by the program logic and the OS.

Unix Processes: notion of **threads** (lightweight processes)

State of an execution e at clock tick i :

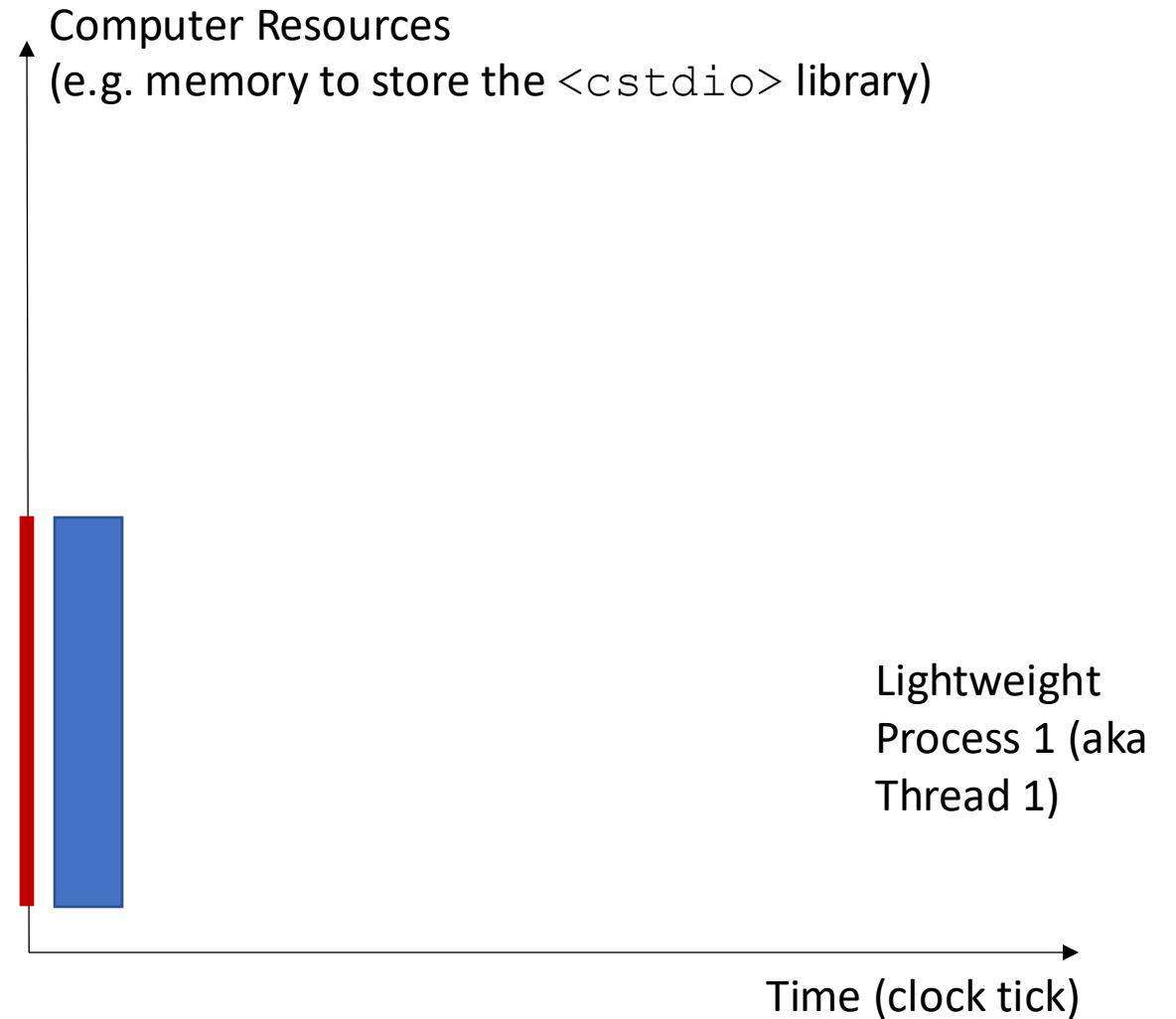
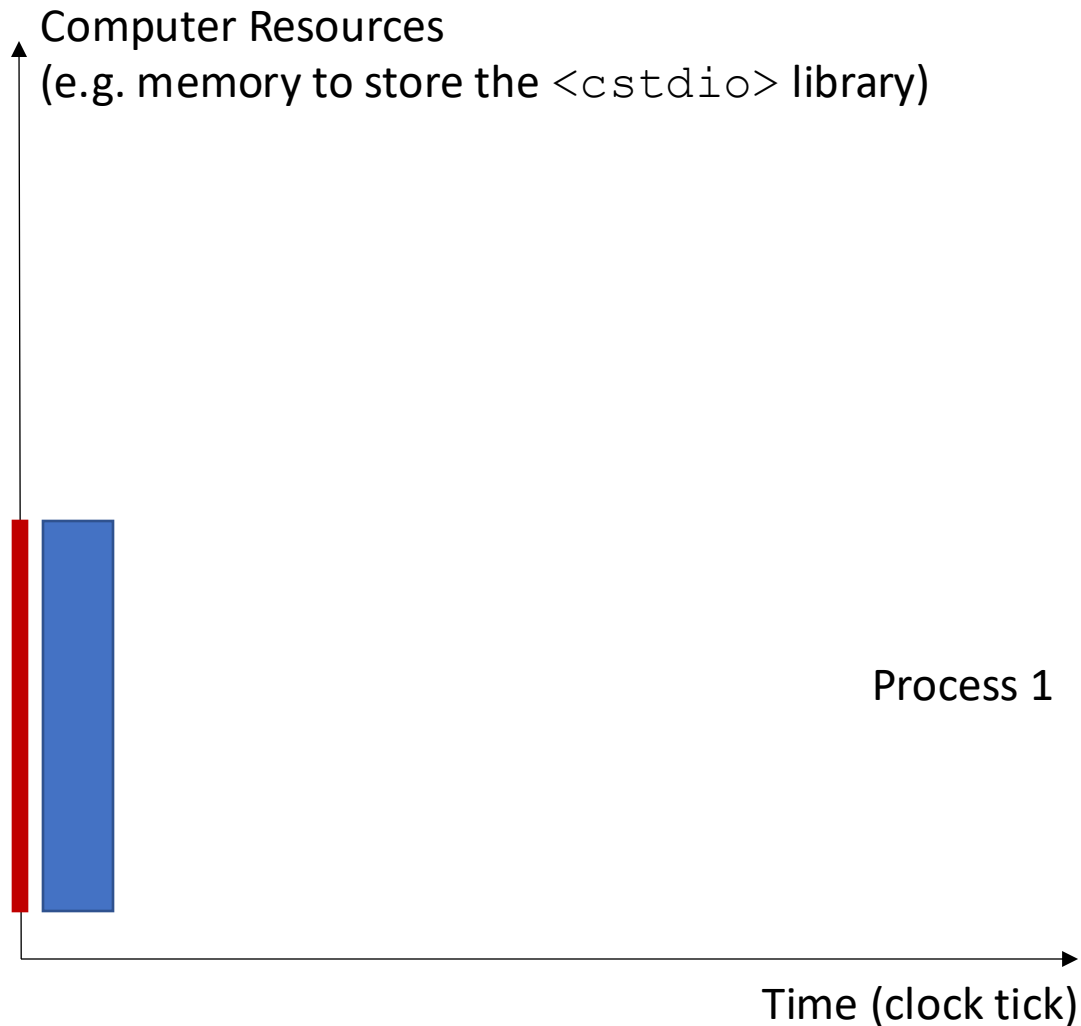
$s(e, i) = (\text{resources used by } e \text{ at } i, \text{state of the resources used by } e \text{ at } i).$

A **process** is the time sequence of $\{s(e, i)\}$, where every two consecutive states $s(e, j)$ and $s(e, j+1)$ have causal relationship determined by the program logic and the OS.

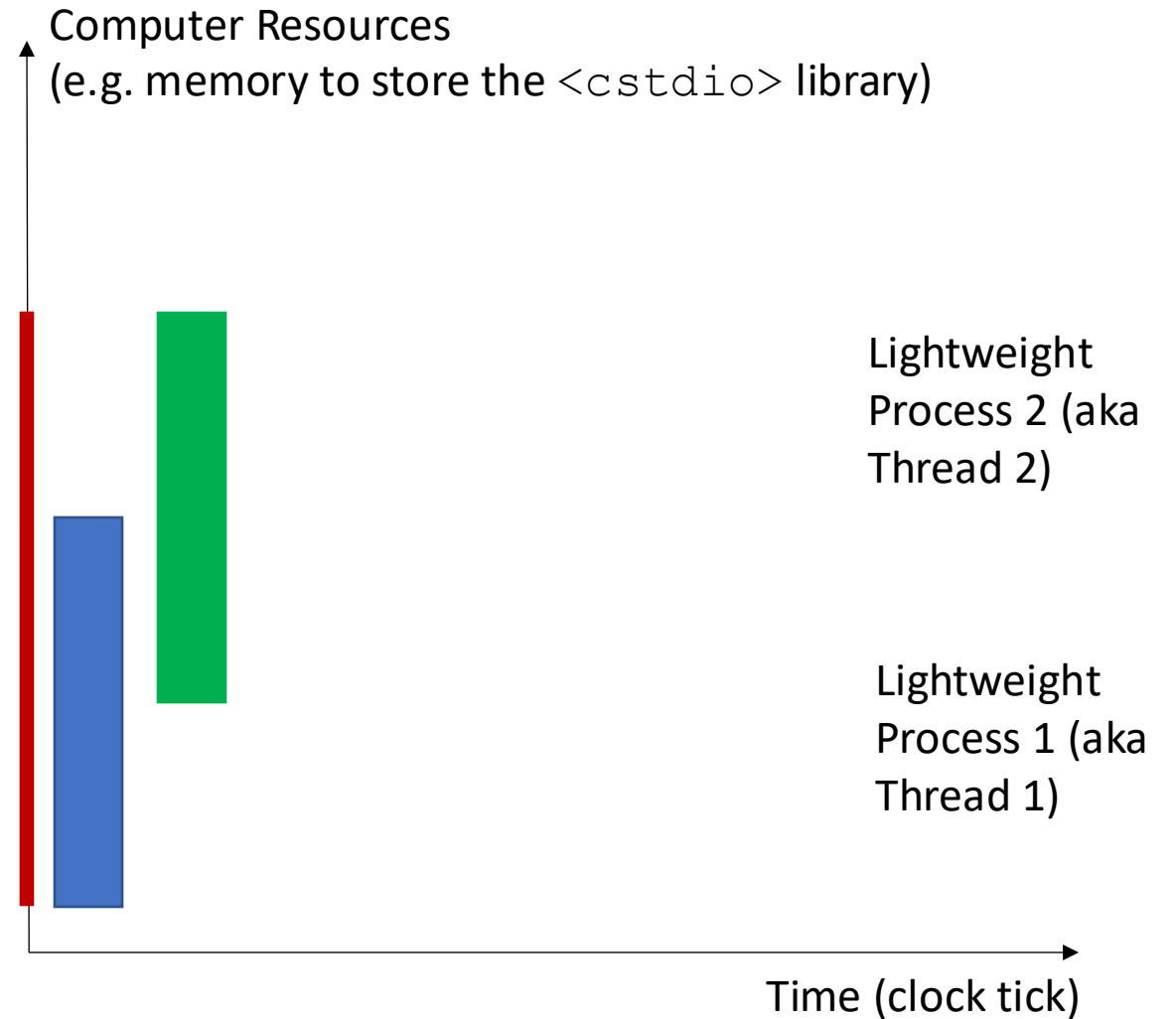
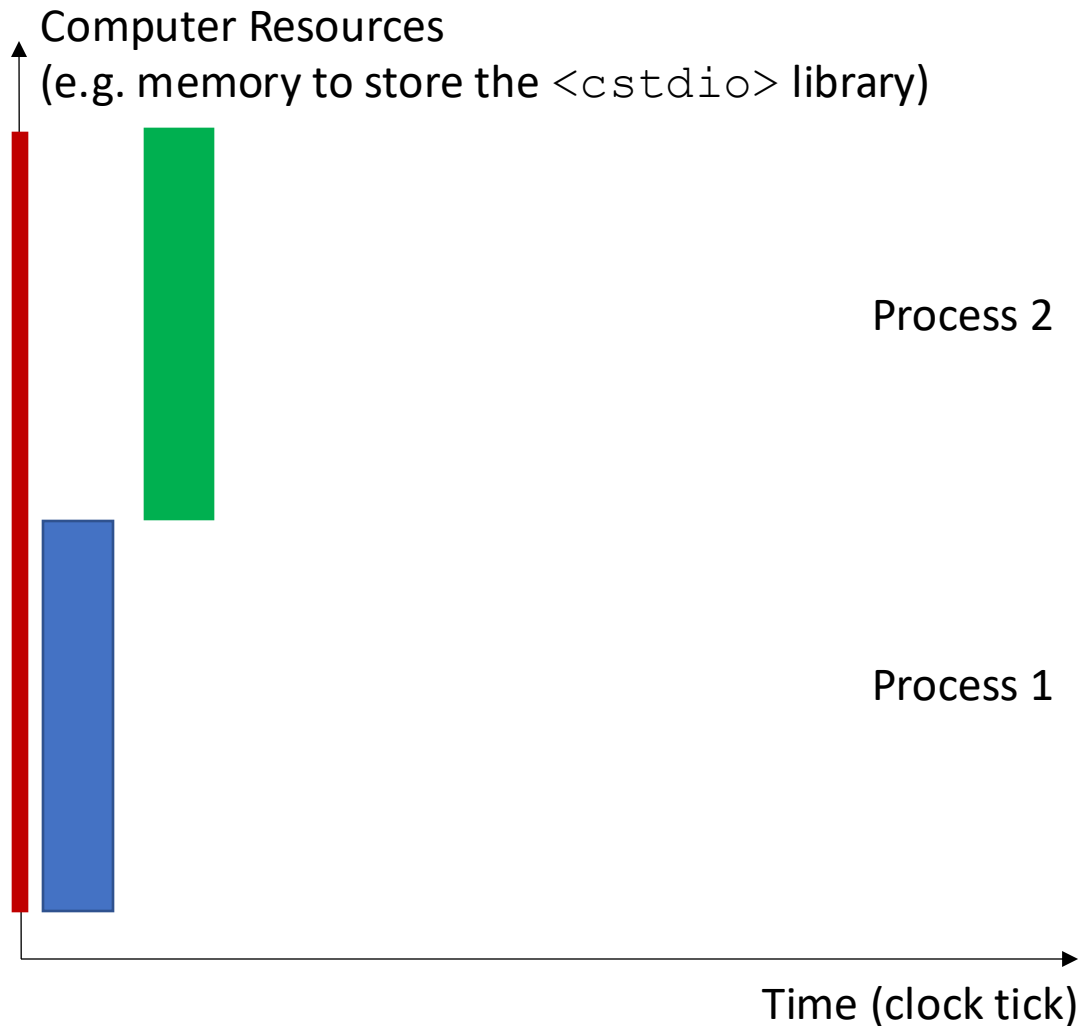
In older OSs, each process has its **exclusive** set of resources, even for processes that are related (e.g. sharing data). This is wasteful.

Modern OSs propose the notion of "threads". Threads are processes that have **shared resources**, typically created by a same **ancestor process**. Because the resource allocation is more thrifty, threads are also called "**lightweight processes**".

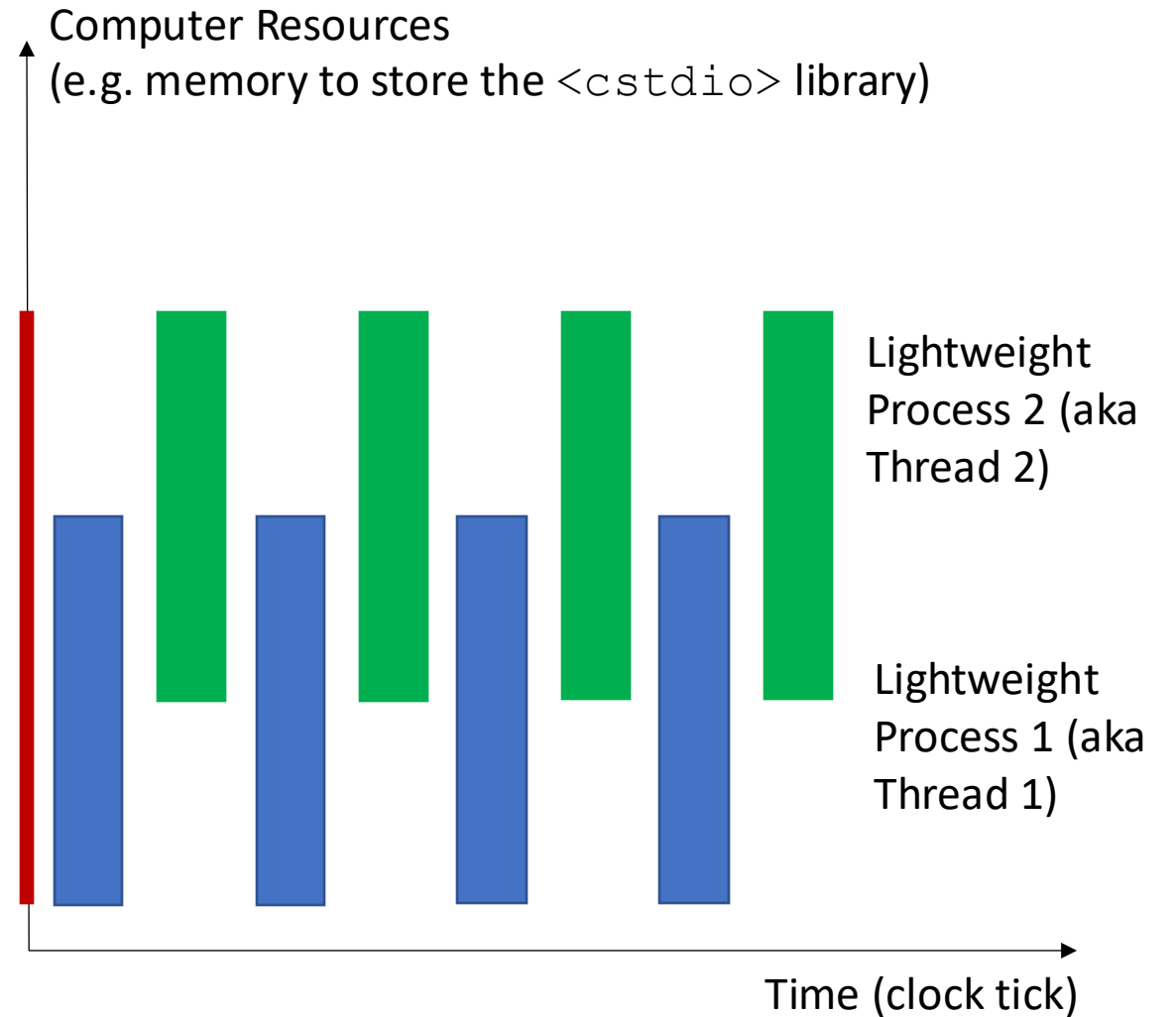
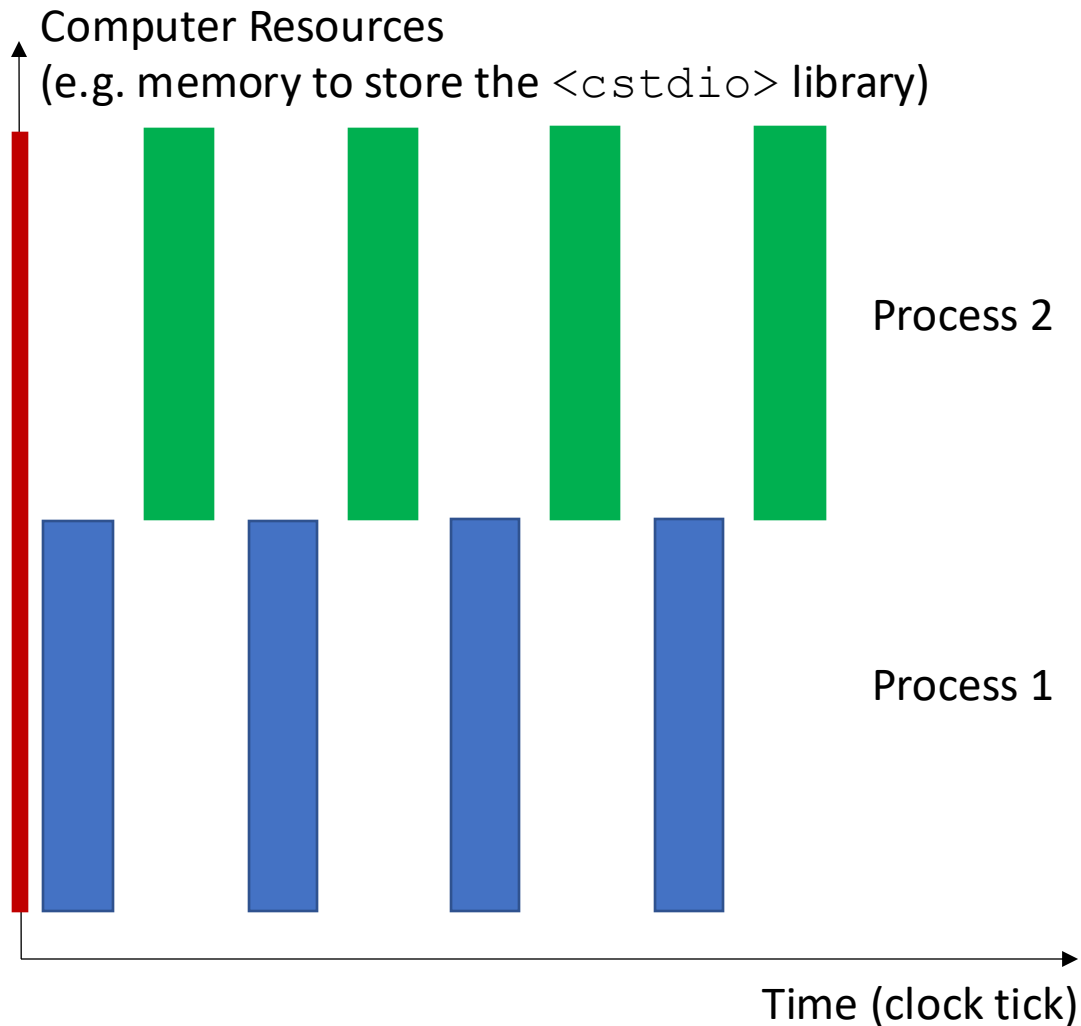
Unix Processes: notion of **threads** (lightweight processes)



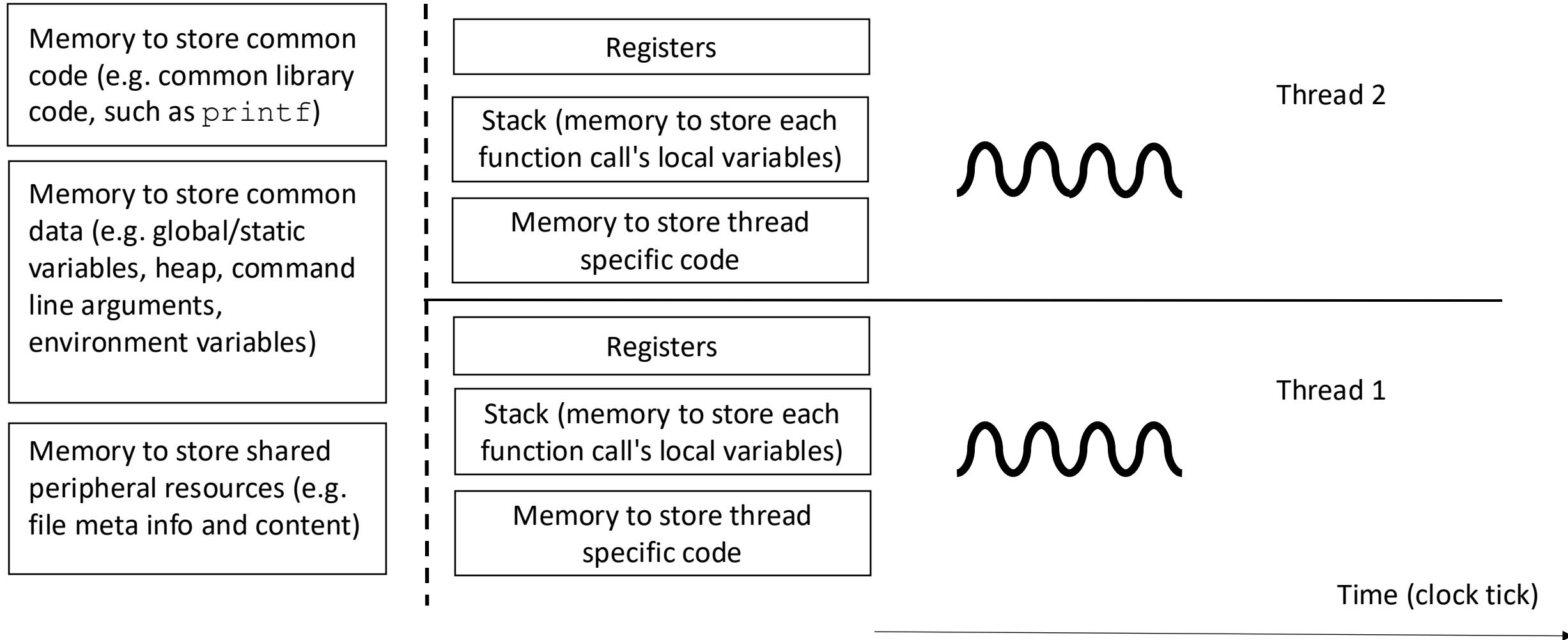
Unix Processes: notion of **threads** (lightweight processes)



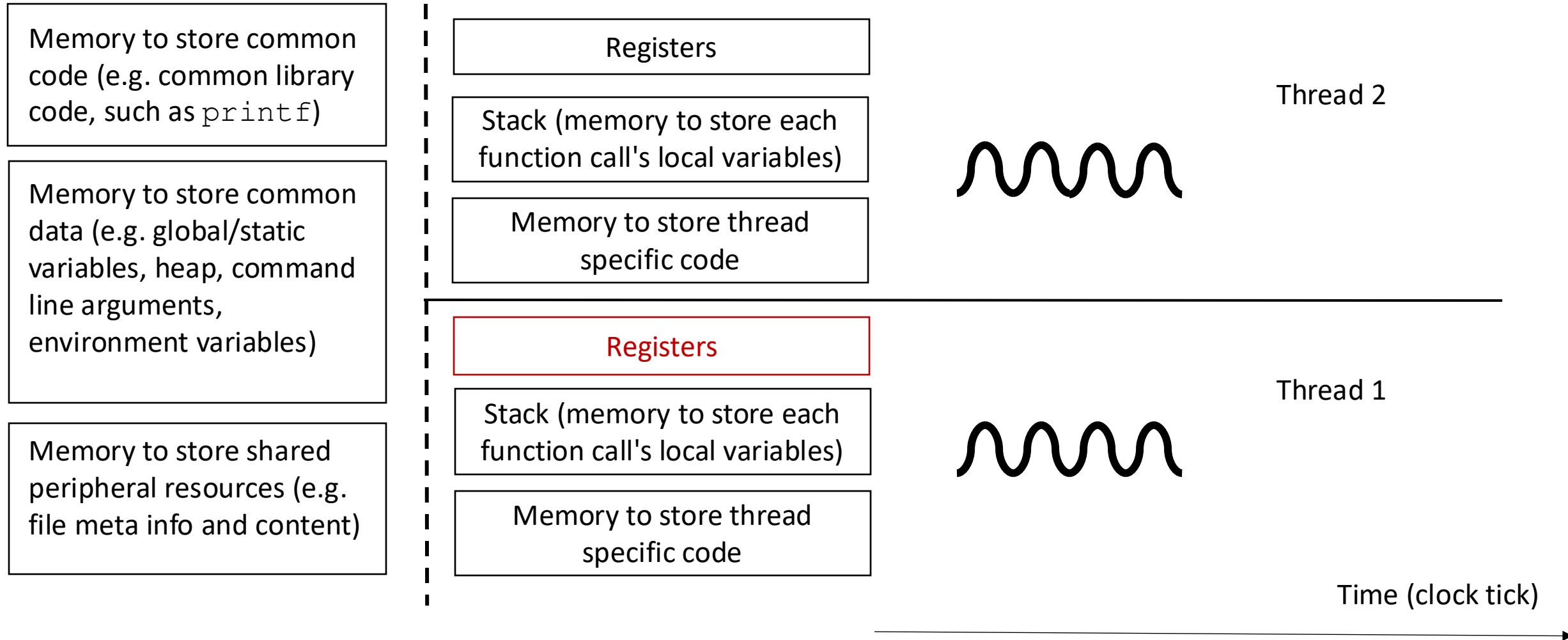
Unix Processes: notion of **threads** (lightweight processes)



Unix Processes: what resources to share and what not to share?

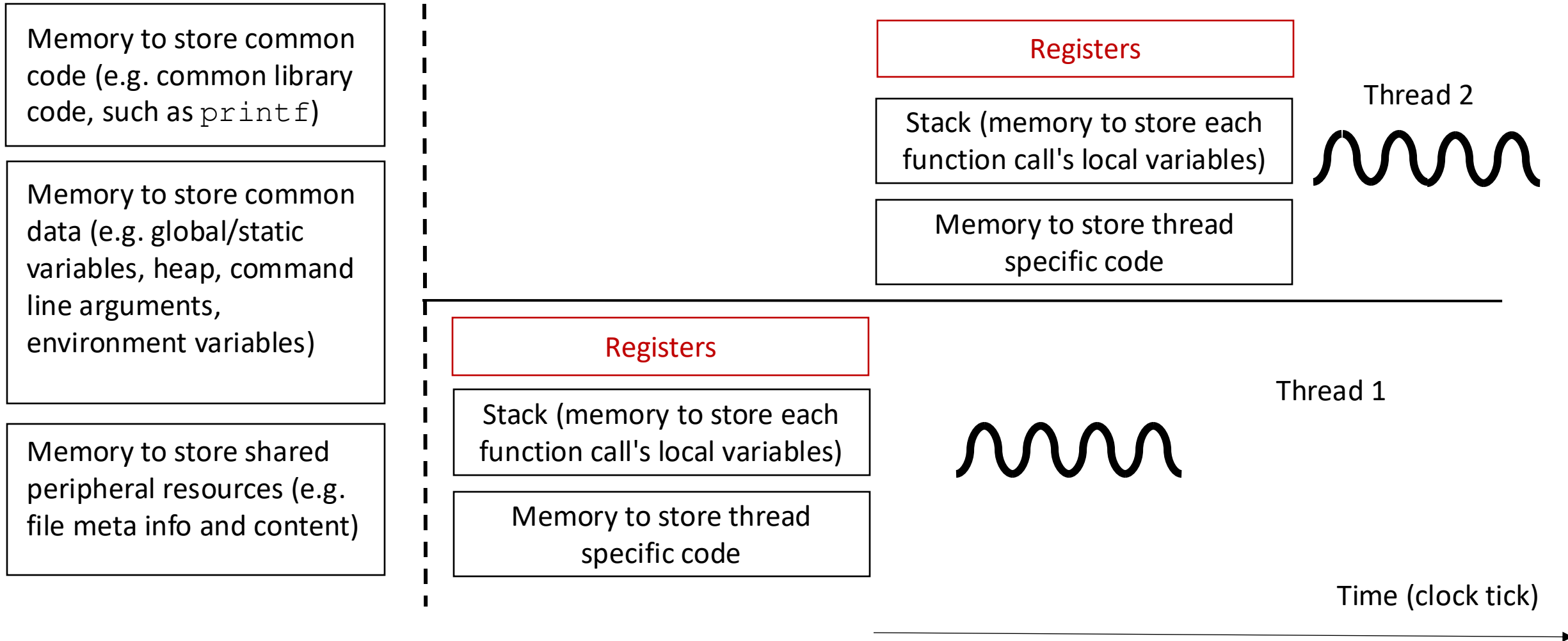


Unix Processes: what resources to share and what not to share?



How to partition registers?

Unix Processes: what resources to share and what not to share?



How to partition registers?

Unix Processes: thread realization and programming interface

All the previous slides on thread are just conceptual guidelines.

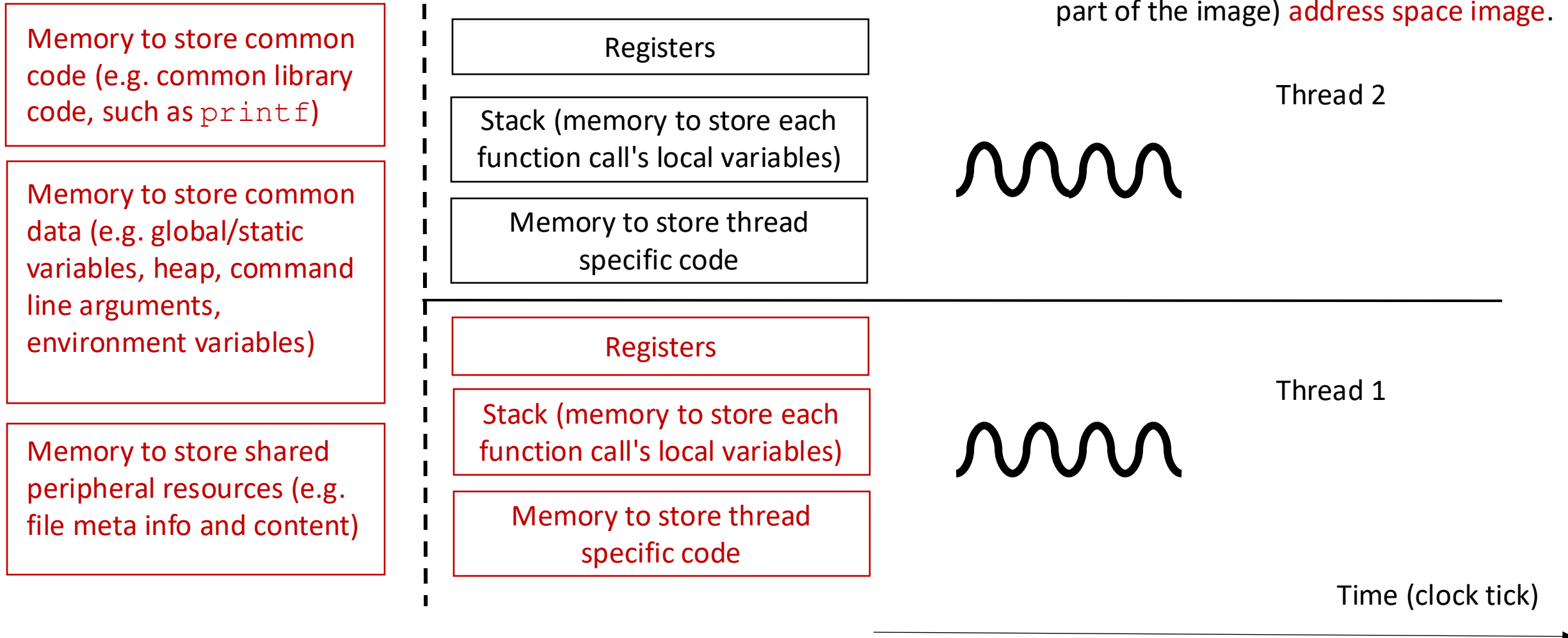
And at the user level, there is a de facto standard C thread programming interface: **PThread** (POSIX Thread).

But different OSs (even OSs belonging to the Unix family) have different ways to realize the concepts and the PThread programming interface.

For example, modern Linux no longer differentiate threads and processes, all are realized as copy-on-write (**COW**) processes (aka "**task**"). For example, every time the parent thread creates a new thread, in the Linux implementation, there is a parent COW process creating a child COW process.

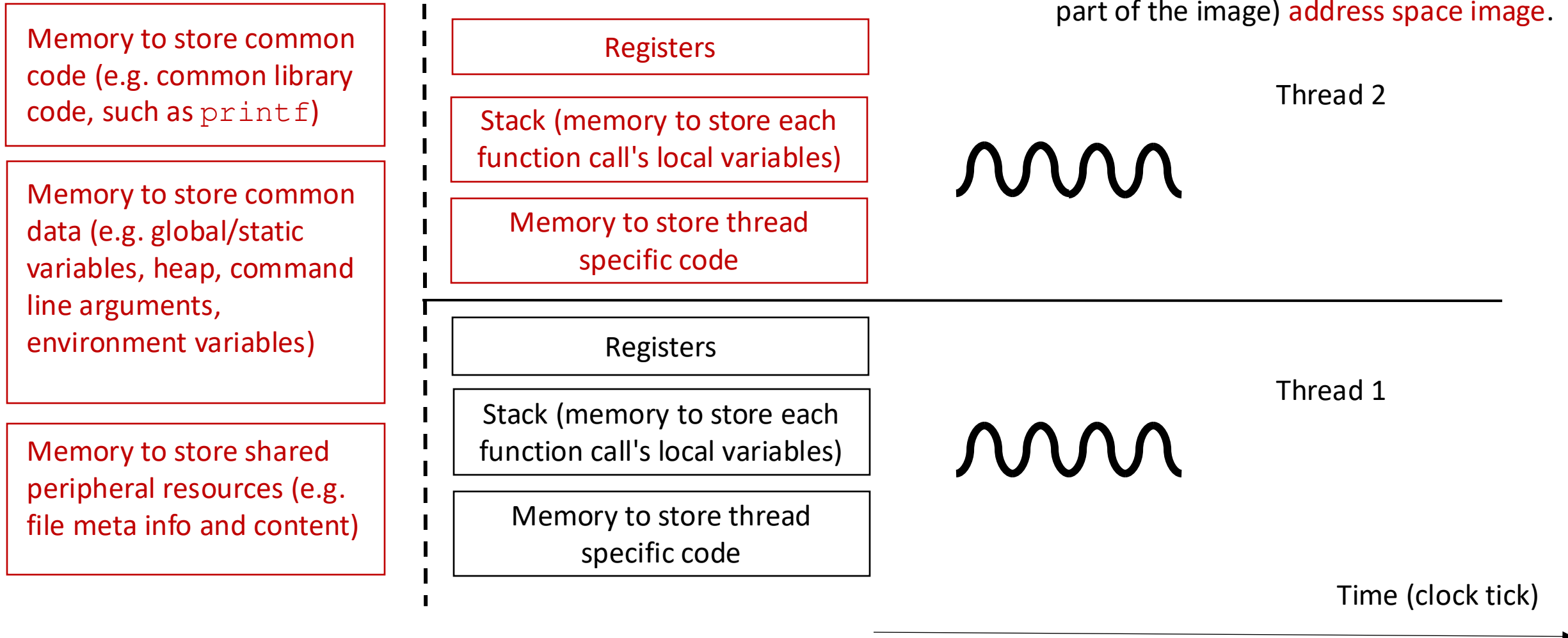
Unix Processes: Process Image

A snapshot of the current state of the process is called a process **image**, aka the process **execution environment image**, or (mainly refers to the memory part of the image) **address space image**.



Unix Processes: Process Image

A snapshot of the current state of the process is called a process **image**, aka the process **execution environment image**, or (mainly refers to the memory part of the image) **address space image**.



Unix Processes: process management

Besides its image, a process has its corresponding meta data (in kernel) for the OS to manage it, e.g. **process ID**.

A Unix OS typically maintains a hierarchy of processes related by parent-child links: the process that requests to create another process is called the **parent** process, the created process is called the **child** process.

A child process inherits all the properties of its parent when it is created, but can change after creation (even changing its code image).

Unix Processes: common process management meta info

Process ID – integer PID

Parent process ID – an integer PPID

User process ID – an integer UID

In Unix, each user has a unique user ID.

Each process is associated with a particular user called the owner of the process, which executes the program.

The owner has certain privileges with respect to the process.

Use `getpid`, `getppid`, and `getuid` to determine the ID of the current, parent, and the owner.

Unix Processes: commands to list these management meta info

`ps` is the short for "process status".

"`ps`" lists your current processes

"`ps -a`" lists more processes, including ones being run by other users and at other terminals (but not include the shells)

"`ps -l`" prints longer, more information lines, including UID, PID, PPID, process status, etc.

Demo Task1

Unix Processes: related Portable OS Interface (POSIX) APIs

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main (void){
    printf("Process ID: %ld\n", (long)getpid());
    printf("Parent proces ID: %ld\n", (long)getppid());
    printf("Owner user ID: %ld\n", (long)getuid());
}
```

Note under POSIX, the
returned type `pid_t`
may be either an `int`
or a `long`

Demo Task2

The **Portable Operating System Interface** (POSIX) is a family of standards specified by the IEEE Computer Society for maintaining compatibility between OSs.

Unix Processes: related Portable OS Interface (POSIX) APIs

To create a new process, typically a parent process calls the `fork()` system call, which traps into the kernel and:

- Allocates a new chunk of memory and kernel data structure.

- Copies the parent process's image and kernel data structure into the new process's, with needed modifications (e.g. PID, PPID).

- Adds the new process to the set of "Ready" processes.

- Returns control back to both processes (by setting the program counter in the respective image, and leaving the processes to the scheduler).

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: forking a process

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int ret_from_fork, mypid;

    mypid = getpid();
    printf("Before: my pid is %d\n", mypid);

    ret_from_fork = fork();

    sleep(1);
    printf("After: my pid is %d, fork() said %d\n", getpid(), ret_from_fork);
}
```

Demo Task3

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: forking a process

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int ret_from_fork, mypid;

    mypid = getpid();
    printf("Before: my pid is %d\n", mypid);

    ret_from_fork = fork();

    sleep(1);
    printf("After: my pid is %d, fork() said %d\n", getpid(), ret_from_fork);
}
```

Example output:

```
>gcc forkdemo1.c -o forkdemo1
>./forkdemo1
Before: my pid is 4170
After: my pid is 4170, fork() said 4171
After: my pid is 4171, fork() said 0
```

Demo Task3

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: forking a process

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int ret_from_fork, mypid;

    mypid = getpid();
    printf("Before: my pid is %d\n", mypid);

    ret_from_fork = fork();

    sleep(1);
    printf("After: my pid is %d, fork() said %d\n", getpid(), ret_from_fork);
}
```

Both the parent and child process continue execution at the instruction after the `fork()`.

But creation of two completely identical processes would not be very useful. We want the parent and child to execute different code.

But how can the parent and child distinguish themselves?

Demo Task3

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: forking a process

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int ret_from_fork, mypid;

    mypid = getpid();
    printf("Before: my pid is %d\n", mypid);

    ret_from_fork = fork();

    sleep(1);
    printf("After: my pid is %d, fork() said %d\n", getpid(), ret_from_fork);
}
```

The value returned by `fork()` allows the parent and the child to distinguish themselves.

`fork()` returns 0 to the child and returns the child's PID to the parent.

Then the child can call `exec` family of functions to execute other programs; it can also retain and keep using the (copied) program code of the parent process.

Demo Task3

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class exercise: forking a process

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int ret_from_fork;
    printf("Before: my pid is %d\n", getpid());

    if ((ret_from_fork = fork()) == 0){
        fprintf(stderr, "I am the child, ID = %ld\n", (long)getpid());
        ...
    }else if (ret_from_fork > 0){
        fprintf(stderr, "I am the parent, ID = %ld\n", (long)getpid());
        ...
    }
}
```

The value returned by `fork()` allows the parent and the child to distinguish themselves.

`fork()` returns 0 to the child and returns the child's PID to the parent.

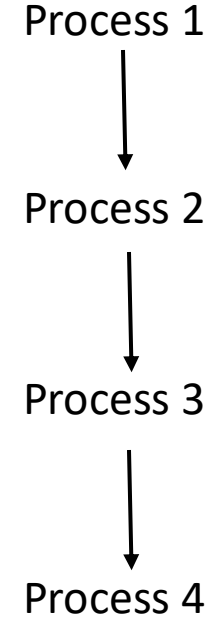
Then the child can call `exec` family of functions to execute other programs; it can also retain and keep using the (copied) program code of the parent process.

Demo Task4

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class exercise: forking a chain of processes

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int i, n = 4;
    pid_t childpid;
    for (i = 1; i < n; ++i)
        if (childpid = fork())
            break; //parent breaks out; child continues
    fprintf(stdout, "This is process %ld with parent %ld, i = %d\n",
        (long)getpid(), (long)getppid(), i);
}
```

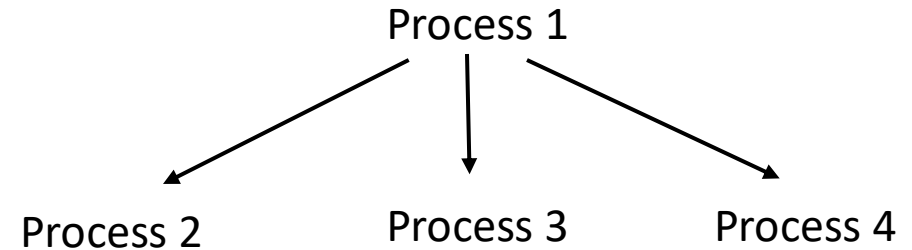


Demo Task5

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class exercise: forking a fan of processes

```
/* forkdemo.c */
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
void main(void){
    int i, n = 4;
    pid_t childpid;
    for (i = 1; i < n; ++i)
        if ((childpid = fork()) <= 0)
            break; //child and error break out; parent continues
    fprintf(stdout, "This is process %ld with parent %ld\n, i = %d",
        (long)getpid(), (long)getppid(), i);
}
```



Demo Task6

Unix Processes: related Portable OS Interface (POSIX) APIs

After fork, both parent and child proceed independently. If a parent wants to wait until the child finishes, it executes `wait` or `waitpid` system call:

```
pid_t wait(int *stat);
```

In general, the `wait` system call causes the caller process to pause until a child terminates or stops, or until the caller receives a signal.

The call returns right away if the process has no children or if a child has already terminated or stopped but has not yet been waited for.

Unix Processes: related Portable OS Interface (POSIX) APIs

If a parent process terminates first without waiting for its children, the children processes become **orphan** processes.

The `stat` is a pointer to an integer variable that stores the **exit status** of the child.

It is almost always poor programming practice to create orphaned processes, because there may be no indication to the user there are still processes running. If one of the orphaned processes gets into an infinite loop, the user may be completely unaware of it; and the orphaned process may continue execution even after the user has logged out.

Unix Processes: related Portable OS Interface (POSIX) APIs

If `wait` returns because a child terminated, the return value is positive and is the PID of that child

Otherwise, `wait` returns `-1` and set `errno`.

`errno == ECHILD` indicates that there were no unwaited-for child processes;

`errno == EINTR` indicates that the call was interrupted by a signal;

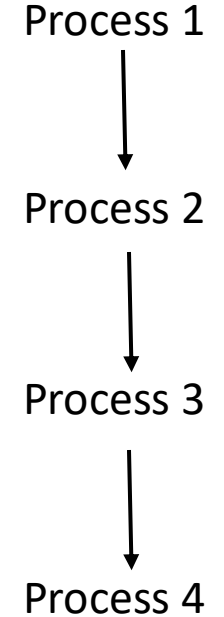
Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: `wait()` example with process chain

Only one forked process is a child of the original process.

Demo Task7

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <errno.h>
void main(void){
    int i, n = 4, status;
    pid_t childpid, waitreturn;
    for (i = 1; i < n; ++i)
        if (childpid = fork()) break; //parent breaks
    while (childpid != (waitreturn = wait(&status)))
        if ((waitreturn == -1) && (errno != EINTR))
            break;
    fprintf(stdout, "I am process %ld, my parent is %ld\n",
            (long)getpid(), (long)getppid());
}
```



Unix Processes: related Portable OS Interface (POSIX) APIs

The `fork` system call creates a copy of the calling process. However, many applications require the child process to execute code **different** from the parent's.

The **`exec`** family of system calls provides a facility for overlaying the calling process with a new executable module

`exec` loads a new executable and arguments into the process image, and calls the `main(...)` function.

If successful, `exec` never returns; the calling process is completely overlaid by the new program and is started from its beginning.

The traditional way to use the `fork-exec` combination is to have the child execute the new program while the parent continues to execute the original code.

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: `exec` example on creating a process to run `ls -l`

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
void main(void){
    int status;
    pid_t childpid;
    if ((childpid = fork()) == -1){
        perror("Error in the fork");
        exit(1);
    }else if (childpid == 0){
        /* child code */
        if (execl("/user/bin/ls", "ls", "-l", NULL) < 0){
            perror("Exec of ls failed");
            exit(1);
        }
    }else if (childpid != wait(&status))
        /* parent code */
        perror("A signal occurred before child exited");
    exit(0);
}
```

Demo Task8

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: `exec` example on creating a process to run `ls -l`

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
void main(void) {
    int status;
    pid_t childpid;
    char *argv[3];
    argv[0] = "ls";
    argv[1] = "-l";
    argv[2] = 0; //NULL pointer
```

```
if ((childpid = fork()) == -1) {
    perror("Error in the fork");
    exit(1);
} else if (childpid == 0) {
    /* child code */
    if (execvp("ls", argv) < 0) {
        perror("Exec of ls failed");
        exit(1);
    }
} else if (childpid != wait(&status))
    /* parent code */
    perror("A signal occurred before child exited");
exit(0);
}
```

Demo Task9

Unix Processes: related Portable OS Interface (POSIX) APIs

Six variations of the exec system call:

```
int execl(const char *path, const char *arg0, ...);
int execlp(const char *file, const char *arg0, ...);
int execl_e(const char *path, const char *arg0, ..., char * const envp[]);
int execv(const char *path, char * const argv[]);
int execvp(const char *file, char * const argv[]);
int execvpe(const char *file, char * const argv[], char * const envp[]);
```

`path`: C string (a NULL ended array of `char`) representing the full path of the executable file

`file`: C string representing only the file name of the executable file, the path is searched from the `PATH` environment

`arg0`: should be the same as file name, the 0th argument passed to `main`.

`...`: a list of `const char *` pointers, each points to a C string, as arguments passed to `main`; the list must end with a `NULL`.

`argv`: an array of `char *` pointers, each points to a C string, as arguments passed to `main`.

`envp`: an array of `char *` pointers, each points to a C string of format "environment_variable_name=value".

Unix Processes: related Portable OS Interface (POSIX) APIs

Six variations of the exec system call:

```
int execl(const char *path, const char *arg0, ...);
int execlp(const char *file, const char *arg0, ...);
int execle(const char *path, const char *arg0, ..., char * const envp[]);
int execv(const char *path, char * const argv[]);
int execvp(const char *file, char * const argv[]);
int execvpe(const char *file, char * const argv[], char * const envp[]);
```

l: arguments are passed to `main` as a list of `const char *` pointers, each pointing to a C string; the list must end with a `NULL`.

v: arguments are passed to `main` as an array of `char * const` pointers, each pointing to a C string.

e: an array of environment variable "environment_variable_name=value" pairs are explicitly passed to the new process image.

p: use the `PATH` environment to search for the executable file.

Unix Processes: related Portable OS Interface (POSIX) APIs

Six variations of the exec system call:

```
int execl(const char *path, const char *arg0, ...);
int execlp(const char *file, const char *arg0, ...);
int execl(const char *path, const char *arg0, ..., char * const envp[]);
int execv(const char *path, char * const argv[]);
int execvp(const char *file, char * const argv[]);
int execvpe(const char *file, char * const argv[], char * const envp[]);
```

An opened file descriptor remains open in the new process image, unless was `fcntl`d with `FD_CLOEXEC` or opened with `O_CLOEXEC`. This is how `stdin`, `stdout`, and `stderr` are opened for the child process.

Unix Processes: related Portable OS Interface (POSIX) APIs

In-class demo: `exec` has no return

File `oldimage.c`

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    printf("Old image: pid=%d\n", getpid());
    execlp("./newimage", "newimage", NULL);
    printf("Old image: hello\n");
    return 0;
}
```

File `newimage.c`

```
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    printf("New image: pid=%d\n",
          getpid());
    return 0;
}
```

Experiment:

```
> ./oldimage
Old image: pid=12369
New image: pid=12369
```

Demo Task10

Unix Processes: process termination

Upon termination of a process, the OS de-allocates the resources held by the process, updates the appropriate statistics, and notifies other processes, specifically these include:

- cancel pending timers and signals;
- release virtual memory spaces;
- release locks, closing open files;
- notifying the parent in response to a `wait` system call.

What happens if the parent is not currently executing a `wait()`?

The child process's kernel management meta info (e.g. task struct) will remain there, though the child process image is gone. The child process hence becomes a **zombie**.

Unix Processes: process termination

A normal termination occurs if there was

- A `return` from `main`,
- An implicit `return` from `main`,
- A call to the C function `exit`, or
- A call to the `_exit` system call

`exit` calls user-defined exit handlers and may provide additional cleanup before it invokes the `_exit` system call.

`exit` takes a single, integer argument, called **exit status**, which will be made available to the parent process (which may be waiting). By convention, a zero means success, some non-zero value means something has gone wrong.

Unix Processes: abnormal process termination

A process can terminate abnormally by

- calling `abort`, causing the `SIGABRT` signal to be sent to the calling process, or
- processing a signal that causes termination.

A code dump may be produced.

User-installed exit handlers will not be called upon abnormal termination.

Unix Processes: background processes

Recall that the shell is a command interpreter which

- Prompts for commands.
- Reads the commands from standard input, forks children to execute the commands, and waits for the children to finish.
- A user can terminate execution of a command by `ctrl-c`.

Most shells interpret a command line ending with `&` as one that should be executed by a background process

When a shell creates a background process, it does not wait for the process to complete before issuing a prompt and accepting additional commands.

Cannot send `ctrl-c` to a background process via the command input. Try

```
tail -f test.txt
```

versus

```
tail -f test.txt &
```

Demo Task 11

Unix Processes: daemons

A daemon is a background process that normally runs indefinitely (have an infinite loop).

Unix relies on many daemon processes to perform routine tasks

- `pageout` daemon handling paging,
- `in.rlogind` handling remote login requests,
- The web server daemon receiving http connection requests
- ftp daemon, mail daemon, ... etc.

How does a daemon process work?

First, edit `/etc/systemd/logind.conf` , set `KillUserProcesses=no`

Second, see Demo Task 12.

Unix Processes: daemons

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <syslog.h>
#include <string.h>

#define MAX_I 100

void main (void){
    pid_t pid, sid;
    FILE* p_output;
    int i = 0;

    pid = fork();
    if (pid < 0) exit(EXIT_FAILURE);
    if (pid > 0) exit(EXIT_SUCCESS);
```

Demo Task12

```
umask(0);

sid = setsid();
if (sid < 0) exit(EXIT_FAILURE);

if ((chdir(".")) < 0) exit(EXIT_FAILURE);

close(STDIN_FILENO);
close(STDOUT_FILENO);
close(STDERR_FILENO);

while(1){
    if ((p_output = fopen(
        "daemon_output.txt", "a")) != NULL){
        fprintf(p_output, "%d\n", i++);
        i %= MAX_I;
        fclose(p_output);
    }
    sleep(3);
}
exit(EXIT_SUCCESS);
}
```